

CI/CD Pipeline in Azure DevOps using Self-Hosted Agent

Introduction

This document explains how to implement a **basic CI/CD pipeline in Azure DevOps** using a **self-hosted agent**.

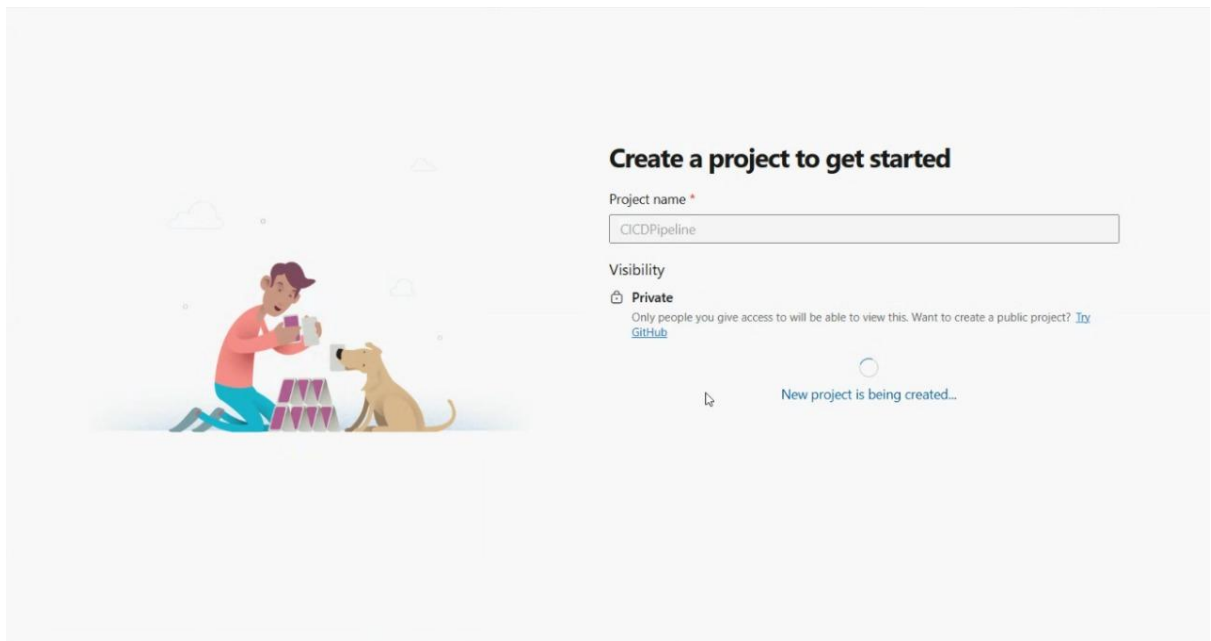
The pipeline executes a simple Python script (hello.py) every time new code is pushed to the repository.

Using a self-hosted agent removes the dependency on Microsoft's hosted parallelism and allows us to run builds on our own machine.

Prerequisites

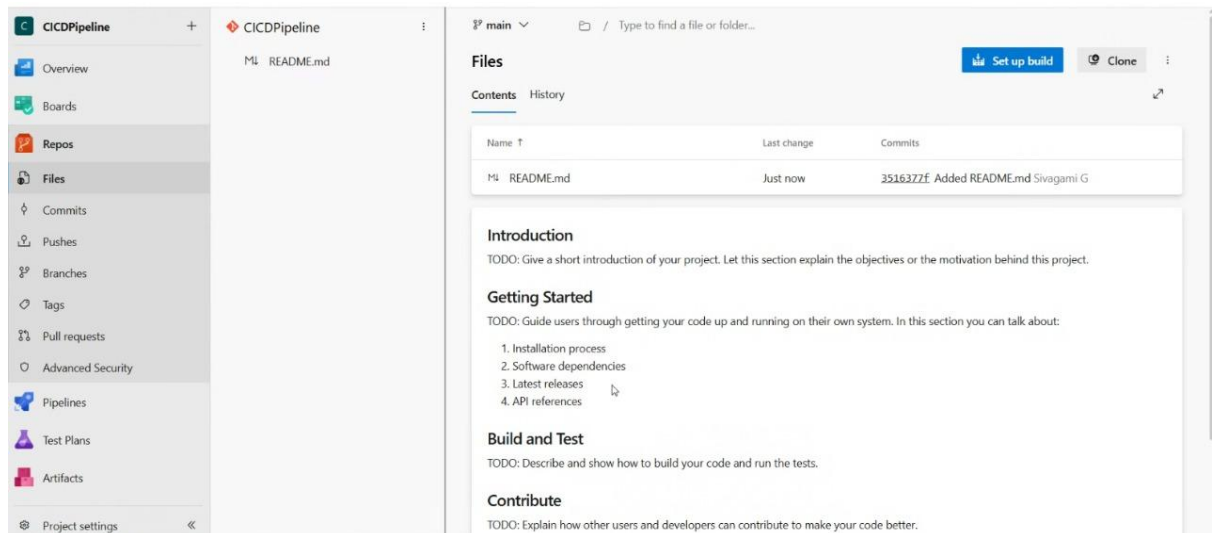
Before starting, ensure you have:

- An **Azure DevOps organization** (<https://dev.azure.com>)
- A project created inside Azure DevOps (e.g., CI-CD-Demo)
- Python installed locally on your machine
- Git installed (optional, if pushing code from local machine)
- A **Personal Access Token (PAT)** in Azure DevOps (used to connect self-hosted agent)

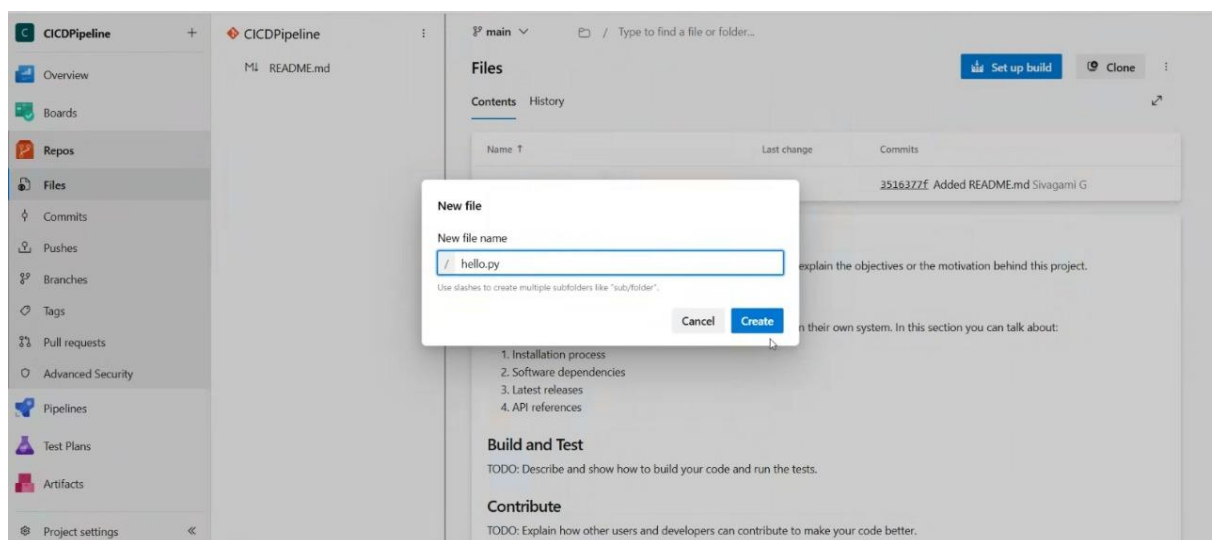


Step 1: Create Repository and Add Python Script

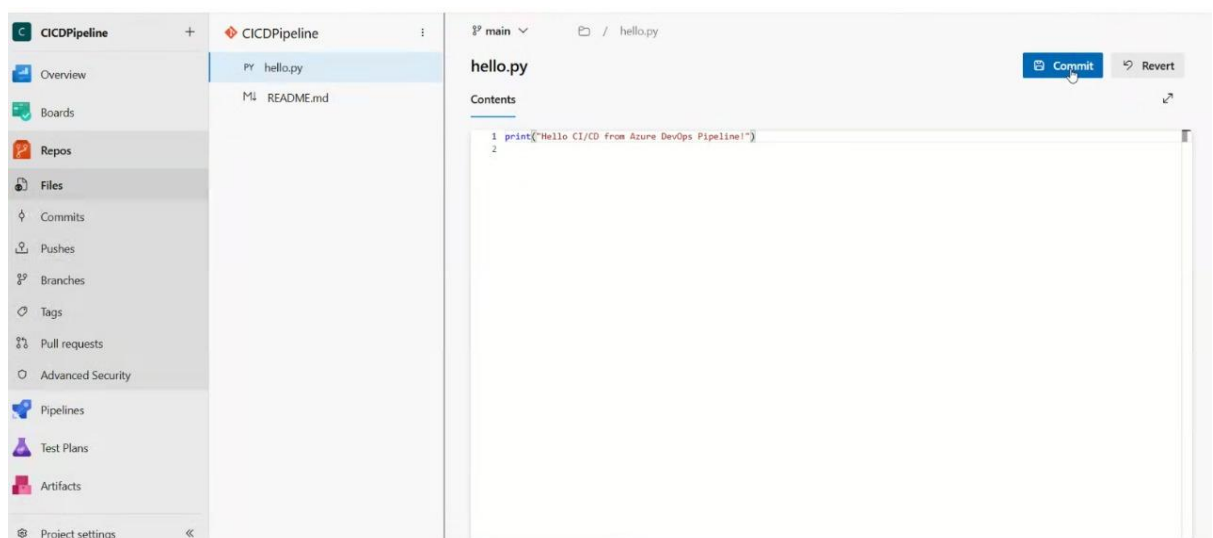
1. Go to your Azure DevOps project → **Repos** → **Files**.
2. If repo is empty, click **Initialize repository**.



3. Create a new file `hello.py` with the following content:



```
print("Hello CI/CD from Azure DevOps Pipeline!")
```



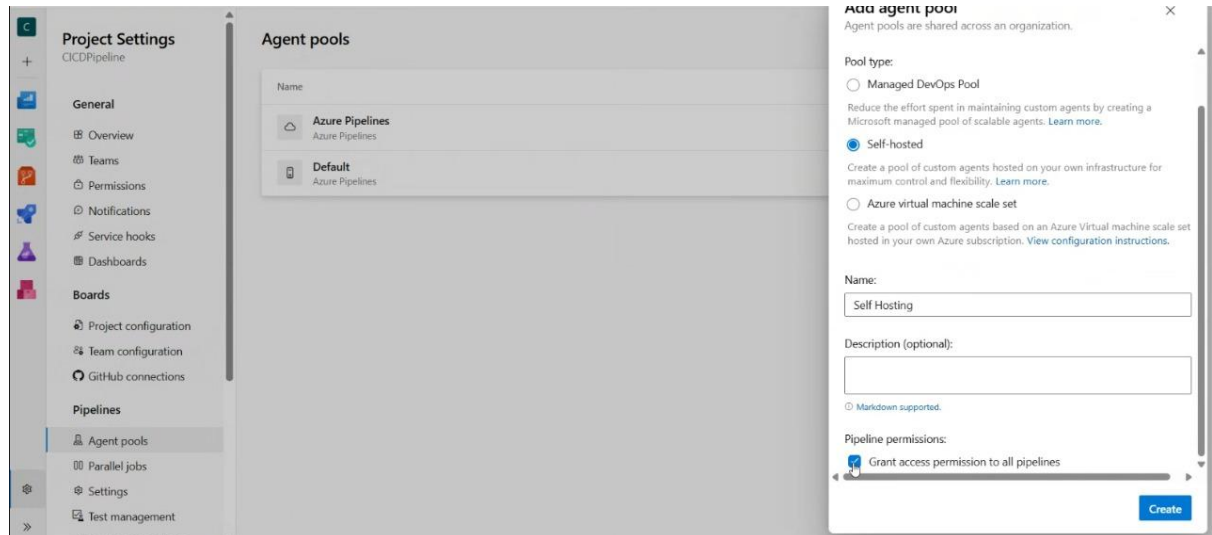
4. Commit the file to the **main** branch.

Step 2: Set Up a Self-Hosted Agent

Since Microsoft-hosted parallelism may not be available by default, we configure our own machine as the build agent.

2.1 Create an Agent Pool

1. Go to **Project Settings** → **Agent Pools**.



2. Click **Add Pool** → Name it SelfHosting.

2.2 Download and Configure Agent

1. Open your new Agent Pool (SelfHosting) → Click **New Agent**.
2. Select your OS (e.g., Windows).
3. Download the agent package (e.g., vsts-agent-win-x64-3.x.x.zip).
4. Extract it to a directory, e.g., C:\agent.

2.3 Configure the Agent

1. Open **Command Prompt as Administrator**.
2. Navigate to the extracted folder:
3. `cd C:\agent`
4. `config.cmd`
5. Provide details when prompted:
 - **Server URL** → `https://dev.azure.com/<your_org_name>`
 - **Authentication** → Choose **PAT**
 - Generate PAT in Azure DevOps: User Settings → Personal Access Tokens → New Token (scope: Agent Pools → Read & Manage).
 - **Agent Pool** → Enter SelfHosting

- **Agent Name** → Example: MyAgent
- **Work folder** → Press Enter for default _work.

```

C:\Windows\system32\cmd.exe: X + v
AZURE PIPELINES
agent v4.260.0 (commit 531b769)

>> Connect:
Enter server URL > https://dev.azure.com/sivagamisiva774
Enter authentication type (press enter for PAT) >
Enter personal access token > ****
Connecting to server ...

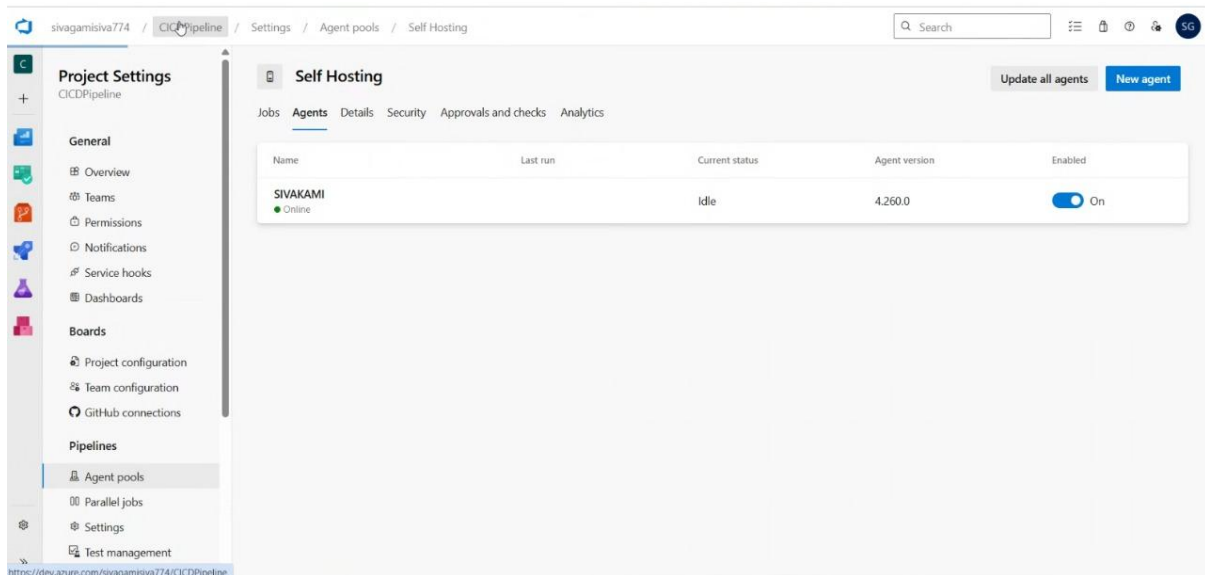
>> Register Agent:
Enter agent pool (press enter for default) > Self Hosting
Enter agent name (press enter for SIVAKAMI) >
Scanning for tool capabilities.
Connecting to the server.
Successfully added the agent
Testing agent connection.
Enter work folder (press enter for _work) > |

```

2.4 Run the Agent

Start the agent interactively:

run.cmd



Now your agent is **online** and connected to Azure DevOps.

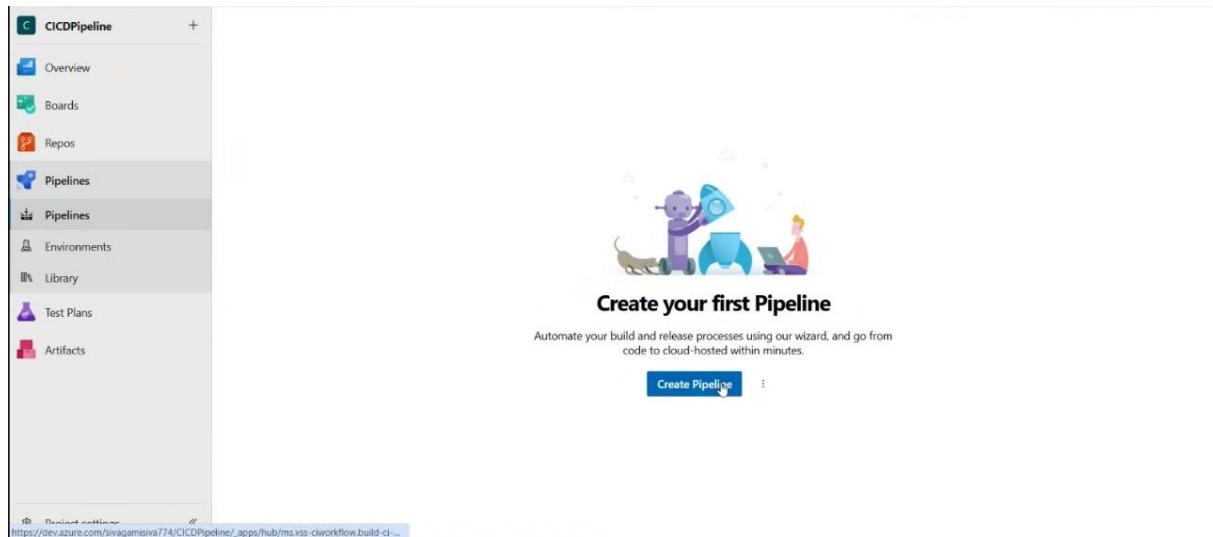
To make it run automatically after reboot:

svc install

svc start

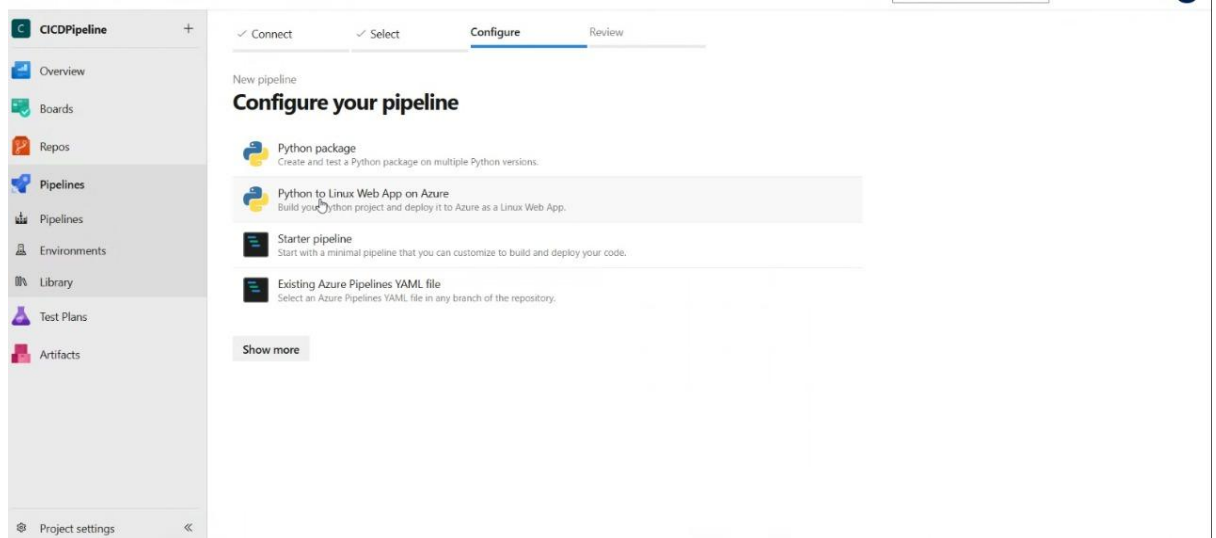
Step 3: Create the Pipeline

1. Go to **Pipelines** → **New Pipeline**.



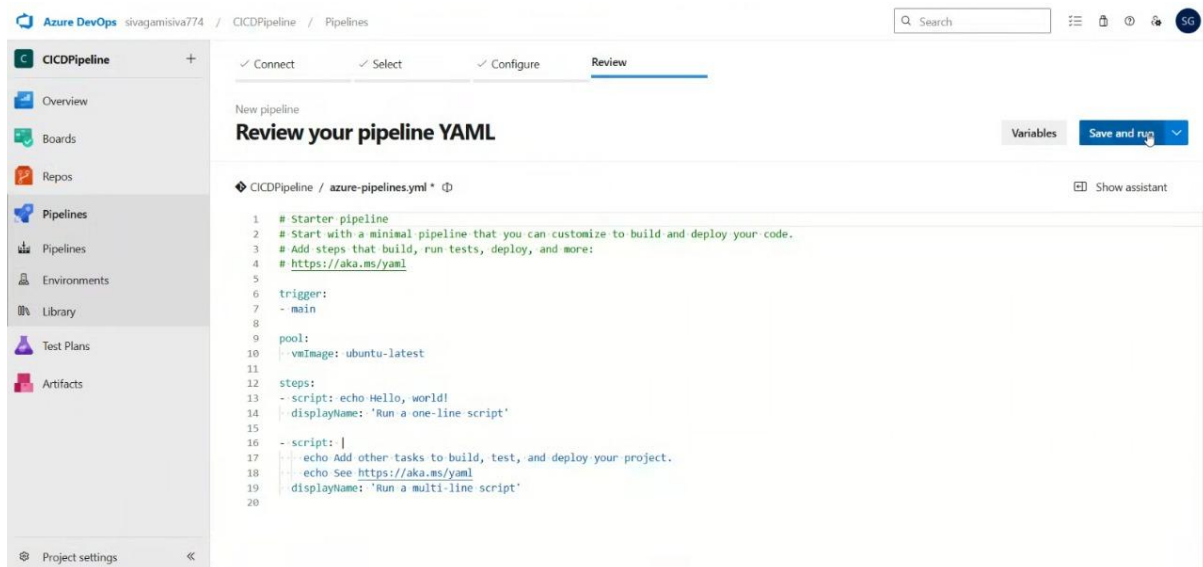
2. Select **Azure Repos Git** → Choose your repo.

3. Select **Starter** pipeline (YAML).

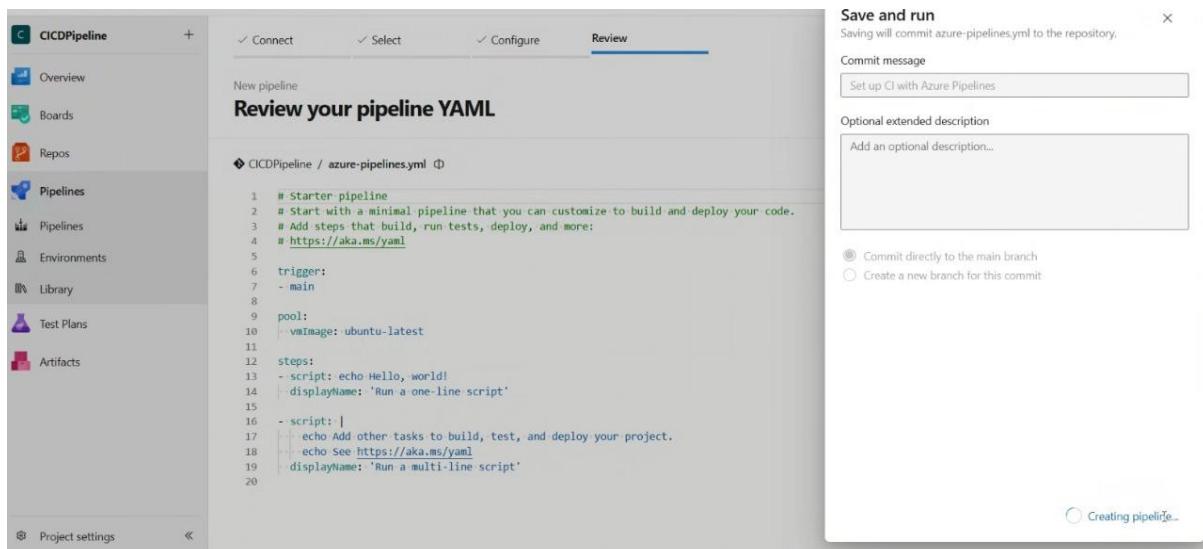


4. Replace the contents with:

```
trigger:
- main    # pipeline runs whenever code is pushed to main branch
pool:
  name: 'SelfHosting'    # use our self-hosted agent pool
steps:
- script: |
    python hello.py
  displayName: 'Run Python script'
```



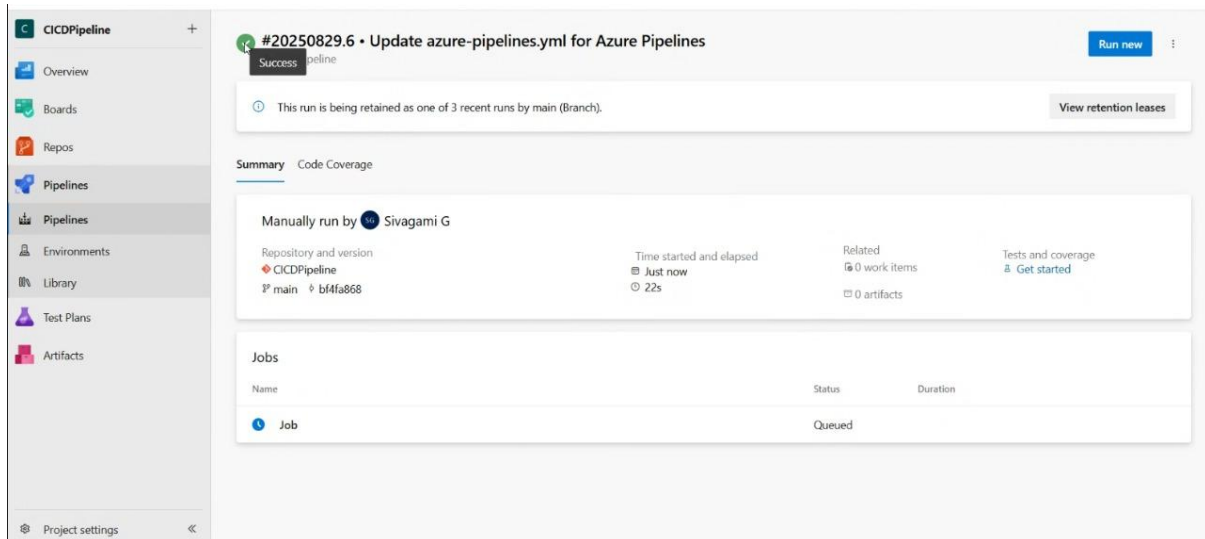
5. Save → This creates azure-pipelines.yml in your repo.



Step 4: Run and Verify Pipeline

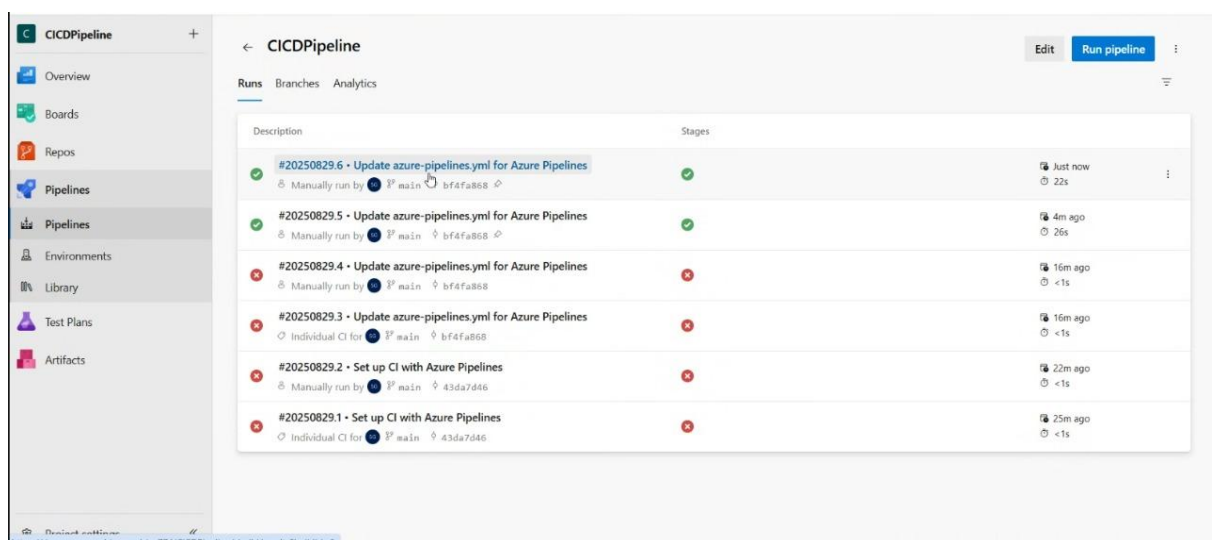
1. Commit a change (or push hello.py).
2. Go to **Pipelines → Runs**.
3. The pipeline should now run on your local machine (self-hosted agent).
4. Check logs for expected output:

```
Hello CI/CD from Azure DevOps Pipeline!
```



Expected Output Screenshots

Screenshot of pipeline run success



Conclusion

This project demonstrates the end-to-end setup of a **CI/CD pipeline in Azure DevOps using a self-hosted agent**.

Every time code is committed to the repository, the pipeline is triggered automatically and runs the Python script on the self-hosted machine.

This provides:

- **Automation** (no manual runs needed)
- **Flexibility** (no dependency on Microsoft-hosted agents)
- **Scalability** (pipelines can be extended with tests, deployments, etc.)